Git Commands from git init to get code of another repo in your local system for command for windows

---

git commands from initialize repo to push

- go your path where you want to initialize empty repository or Code you want to push

- git init [Initialized empty Git repository in] E:/Giri's-Tech-Full-Stack-Java/Core-Java/Oct-Revision-Plan/.git/

  → git status;

  → git add .

  →git commit -m "revision code of Java-HTML-SQL";

  →git remote add origin https://github.com/ABHI7pute0205/Giri-s-TechHub-Task.git

  →git branch -M main

  →git push -u origin main

  → git remote -v
  origin https://github.com/ABHI7pute0205/Giri-s-TechHub-Task.git (fetch)
  origin https://github.com/ABHI7pute0205/Giri-s-TechHub-Task.git (push)

  (after successfully push code file on git you will see like this |)

---

git config --global user.name "Your Name"
git config --global user.email "you@example.com"

Ya main branch var ti ch code push kara y cha asla tar same path (Local repo) chya location var tun hya 3 command follow kara y chya varchya step's complete zalya var in future if we want to add some code on this same location

- git add .
- git commit -m "Updated something"
- git push

Yat Jar aapn Git chya same repository var ti LOCAL (Laptop) Repository madhun jar code push kara y cha asla tar ti repository jar aadhe ch LOCAL madhlya dusry FOLDER la Assign Asle tar Laptop Mapdhlya different - different Folder madhun same repo var ti code push kar ta yet nahi tith MERGED CONFLICT yeto so tya sathi NEW BRANCH create kara y che and tya var ti code push kara y cha git chya same repo var ti and Then later tya branch merged kara y chya

दुसऱ्या project/folder मध्ये git initialize केलेल्या folder मध्ये जा
cd path/to/other/local/project

Git remote add करा (जर आधी नसेल)
git remote add origin
https://github.com/ABHI7pute0205/Giri-s-TechHub-Task.git

 # नवीन branch बनवा
git checkout -b new-branch-name

सर्व फाइल्स stage करा
git add .

commit करा
git commit -m "Add new project code"

Push करा नवीन branch वर
git push -u origin new-branch-name

Local Git मध्ये Merge करून Push

Part jar main branch var ti shift kara y ch asl tar

Main branch वर जा
git checkout main

Local मध्ये नवीन branch merge करा
git merge new-feature-branch
जर conflicts आले, तर फाइल्समध्ये <<<<<<<, =======, >>>>>>> markers दिसतील.
त्यांना manually resolve कर → git add . → git commit -m "Resolved conflicts"

Merge झाल्यावर GitHub वर push करा
git push origin main

==========

GitHub वरून Merge (Pull Request वापरून)

1. GitHub वर जा → तुझ्या repository मध्ये जा
2. वरच्या menu मध्ये **Pull requests** वर क्लिक कर
3. **New Pull Request** बटण क्लिक कर
4. Base branch = `main`
   Compare branch = `new-feature-branch`
5. Differences बघा → Merge करण्यासाठी **Create Pull Request**
6. Review करून **Merge Pull Request** → Confirm merge
   हे method safe आहे आणि conflicts सहज resolve करता येतात.

Git commands to create new file and make change in it and push

---

- Make a new directory (folder) → **mkdir MyProject**
- Check it's created: → **dir**
- Go inside the folder → **cd MyProject**
- Create a new file → **echo.> MyFile.java**
- Open the file in CMD (using Notepad) → **notepad MyFile.java**
- Check Git status (if repo initialized) → **git status**

- Add file to Git staging area → **git add MyFile.java**
- Commit the changes → **git commit -m "Added MyFile.java"**

Steps to Create and Merge a Branch in Git :

→ git branch **(we see the branch in which we are currently)**

→git branch new-feature **(here we create new Branch)**

→git checkout new-feature **(Switch to that new branch)**

 # make Some changes

→ git add . (add those files)

→ git commit -m "message" **(add commit)**

→ git checkout main **(Go back to the main branch)**

→ git merge new-feature **(Merge the new branch into main)**

→ git branch -d new-feature **((Optional) Delete the merged branch)**

→ git push **(Push the updated main branch to GitHub)**

this command is used to **check the number files present on your current gitHub repo → git ls-files**

---

clone a GitHub repository to your local system in Windows terminal :

→ Open Terminal

- You can open:
- Git Bash (recommended) OR
- Command Prompt (cmd)

→ Go to the location where you want to clone the repo
- cd "E:\Giri's-Tech-Full-Stack-Java\Core-Java"

→ Copy the repository URL from GitHub

→ On GitHub:
- Go to your repository page
- Click on the green “Code” button
- Copy the HTTPS URL
- e.g /https://github.com/your-username/your-repo-name.git

→ Clone the repository
- Now in your terminal, run:
- **git clone /https://github.com/your-username/your-repo-name.git**

This command downloads all the code and creates a local copy of the repository.

→ Go to inside the cloned folder
- cd your-repo-name

→ Verify
- dir (for Command Prompt) OR
- ls (for Git Bash)

How to Add remote repository

1️⃣ **Go to your local repository**

cd "E:\Giri's-Tech-Full-Stack-Java\Core-Java\MyProject"

2️⃣ **Initialize Git (if not already done)**

git init

3️⃣ Check current remotes

git remote -v

4️⃣ Add remote repository

git remote add origin https://github.com/YourUsername/YourRepoName.git

Note: `origin` is just a conventional name for the remote. You can use any name.

5️⃣ Verify remote is added

git remote -v

Output should look like:

origin https://github.com/YourUsername/YourRepoName.git (fetch)
origin https://github.com/YourUsername/YourRepoName.git (push)

6️⃣ Push your local code to GitHub

If your branch is `main`:

git push -u origin main

- `-u` → sets upstream so next time you can just do `git push`
- `origin` → remote name
- `main` → branch name

For older repos, the default branch might be `master`. Adjust accordingly

To change remote URL later:

git remote set-url origin https://github.com/YourUsername/NewRepoName.git

To **remove remote**:

git remote remove origin

Other commands of the git are same on all OS Just change the File create open commands based on OS

Git Commands for Ubuntu/Linux OS

Git commands to create a new file and push

Make a new directory:

mkdir MyProject

Check it's created:

ls

Go inside the folder:

cd MyProject

Create a new file:

touch MyFile.java

Open the file using nano editor:

nano MyFile.java

Other ways to open a File in Ubuntu/Linux OS

1️⃣ **Using `vim` editor** (usually pre-installed)

vim MyFile.java

Press `i` to enter **Insert mode** and start typing.

Press `Esc` → type `:wq` → press `Enter` to **save and exit**.

To exit without saving: `:q!`

2️⃣ **Using `gedit` (GUI text editor)**

gedit MyFile.java &

Opens a graphical window.
Use `&` to keep the terminal free.
Useful if you're on Ubuntu Desktop.

3️⃣ **Using `cat` (view only, not for editing)**

```
cat MyFile.java
```

Displays file content in the terminal.
Not editable, just for checking.

4️⃣ **Using `less` (view only, scrollable)**

```
less MyFile.java
```

Scroll with arrow keys.
Press `q` to quit.

5️⃣ **Using `touch` + redirect operators (quick edit in terminal)**

```
echo "public class MyFile {}" > MyFile.java
```

Creates or overwrites files with content.
Not interactive editing, but fast for small edits.

Check Git status:

```
git status
```

Add file to staging:

```
git add MyFile.java
```

Commit changes:

```
git commit -m "Added MyFile.java"
```

Jar git chya repo 2 different - different folder madhe initialize kele asle tar / aani git var ti to code 2 different branch var ti push kele asla AND tya brancgh merge kara y chya asya var ti

---

To merge the code of 2 branch on Git

Step-1 : git checkout main

step-2 : git merge UI-Code-New-Branch

step-3 : git pull origin main --allow-unrelated-histories
(जेव्हा तुझं local repo आणि remote repo अलग-अलग initialized असतात (म्हणजे दोघांचे histories independent असतात) —
तेव्हा Git सामान्यतः merge करण्यास “refuse” करतो, कारण त्याला वाटतं हे दोन्ही वेगळ्या projects आहेत.)
-- Merge remote main with local main (even if both are unrelated) [mhanje he command aapn git jar 2 thika ne init kele l asl aani eka repo madhe

2 branch var ti tya cha code push asla tar to merge kar nya sathi he command]

Step-4 : git add .

Step-5 : git commit -m "merge remote main with local main"

Step-6 : git push origin main

Jar eka ch repository var ti 2 different folder madhun different branch var ti code push kart aslo tar git push hot nahi remote repo madhe so in this case we need to **git pull** this command load all the new merged files into local repo or folder from where you push new code

==========

Merge Two Branch of the git which are in same directory Only From Terminal

Master is the default branch on which we already push some code

To merge two branches need to create another branch other than master

Code to create another branch and Push code on it :

- **git branch subbranch1**
- **git checkout subbranch1**
- **git add filename.txt**
- **git status**
  On branch subbranch1
- **git commit -m "New file is added";**
- **git push -u origin subbranch1**

Commands to merge this 2 branches :
Move to branch on which you want to merge your code

- **git checkout master**
- **git merge subbranch1**
- **git push origin master**
- **git fetch origin**
- **git pull origin master --rebase**
- **git merge master**
- **git push origin master**

After completely merging **delete branch** created by user

E:\Bootsrap >git branch -d subbranch1

warning: deleting branch 'subbranch1' that has been merged to
'refs/remotes/origin/subbranch1', but not yet merged to HEAD
Deleted branch subbranch1 (was 17d2a85).

E:\Bootsrap >git branch
* master
==========

Push Code on **GitLab**

Step-1 : open account on GitLab https://gitlab.com/
Step-2 : Login this account with github we see option below
Step-3 : Create repository on GitLab Login to GitLab → New Project → Name it gitlabproject → Create
Step-4 : Connect local repository with GitLab remote
git remote add origin https://gitlab.com//gitlabproject.git
Step-5 : remaining all commands same like as github check status ,git init , add etc
E.g.

- git init
- git status
- it add .
- git commit -m "add this file on gitLab"
- git remote add origin https://gitlab.com/abhi7pute0205-gitlabgroup/ABHI7pute0205-projectSYMSC.git
- git branch
- git push -u origin master

For creating and merging new branch follow git commands
==========

Push code on **BitBucket** :
https://bitbucket.org/abhisheksatputetest/workspace/overview/

In these when we login then go to the home page and see the recent repository click on it and to copy SHH and HTTP link click on **clone** and copy it.

Slips jya ki aaplya ghya y chya nahi : 12 , 24

To Create Maven .jar file follow this steps and run on terminal : once we create Maven project add dependencies in it and and compile and run then

Download Maven Zip https://maven.apache.org/download.cgi :

Step 1: **pom.xml** मध्ये Main Class सेट करणे
`pom.xml` मध्ये `<build>` टॅगमध्ये plugin add कर:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                            https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>demo.demo1</groupId>
  <artifactId>SyMscCsMavenProject</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <version>1.18.42</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>8.0.33</version>
    </dependency>
  </dependencies>
  <build>
    <plugins>
```

```
        <!--Executable JAR with dependencies-->
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-assembly-plugin</artifactId>
          <version>3.6.0</version>
          <configuration>
            <archive>
              <manifest>
                <mainClass>demo.demo1.TestApp</mainClass>
              </manifest>
            </archive>
            <descriptorRefs>
              <descriptorRef>jar-with-dependencies</descriptorRef>
            </descriptorRefs>
          </configuration>
          <executions>
            <execution>
              <phase>package</phase>
              <goals>
                <goal>single</goal>
              </goals>
            </execution>
          </executions>
        </plugin>
      </plugins>
    </build>
</project>
```

Package ch nav sagli kad same pahije :

C:\Users\abhis\eclipse-workspace\SyMscCsMavenProject>**mvnd --stop**
Stopping 2 running daemons

C:\Users\abhis\eclipse-workspace\SyMscCsMavenProject>**mvnd clean package**

C:\Users\abhis\eclipse-workspace\SyMscCsMavenProject>**cd target**

C:\Users\abhis\eclipse-workspace\SyMscCsMavenProject\target>**java -jar SyMscCsMavenProject-0.0.1-SNAPSHOT-jar-with-dependencies.jar**
Hello This is Maven Project

Junit Dependency For Practicals

```
 <dependencies>
        <dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.10.0</version> <!-- Or the latest stable version -->
  <scope>test</scope>
</dependency>
 <dependency>
   <groupId>org.junit.platform</groupId>
   <artifactId>junit-platform-suite</artifactId>
   <version>1.10.2</version> <!-- or latest -->
   <scope>test</scope>
</dependency>

</dependencies>
```

==========

Steps to download Jenkins :

Go to this link download LTS version for windows
https://www.jenkins.io/download/#downloading-jenkins

Here we have mentioned the steps to install :
https://www.jenkins.io/doc/book/installing/windows/

Default port for the jenkins : 8080

After that finish and install then open browser and type http://localhost:8080

And here enter password from the location

Git All Commands

```
mkdir MyProject
cd MyProject
git init
touch file1.txt
git status
git add .
git commit -m "first commit"
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git remote add origin https://github.com/username/MyProject.git
git branch -M main
git push -u origin main

git branch feature1
git checkout feature1
git add .
git commit -m "feature1 changes"
git push -u origin feature1

git checkout main
git pull
git merge feature1

# If conflict:
# edit file manually
git add .
git commit -m "resolved merge conflict"
git push

# If push error
git pull --rebase
```

git push